

Data Wrangling (Data Preprocessing)

Mid-term Assessment

Wan Yi Lam

30 April 2024

Setup

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.5
## ✓ forcats    1.0.0      ✓ stringr    1.5.1
## ✓ ggplot2    3.5.2      ✓ tibble     3.2.1
## ✓ lubridate  1.9.4      ✓ tidyr      1.3.1
## ✓ purrr      1.0.2
## — Conflicts — tidyverse_conflicts() —
## * dplyr::filter() masks stats::filter()
## * dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(magrittr)
```

```
##
## Attaching package: 'magrittr'
##
## The following object is masked from 'package:purrr':
##
##     set_names
##
## The following object is masked from 'package:tidyr':
##
##     extract
```

```
library(dplyr)
library(knitr)
library(randomNames)
library(Hmisc)
```

```
##
## Attaching package: 'Hmisc'
##
## The following objects are masked from 'package:dplyr':
##
##      src, summarize
##
## The following objects are masked from 'package:base':
##
##      format.pval, units
```

Introduction

Commonwealth Bank of Australia focus on providing both retail and commercial banking services in Australia, head of retail banking division would like to random check about secured and unsecured loans performances of five retail branches in January in order to better understand market trends and customers' need.

In this report, there are two synthetic data sets with 120 observations created; one contains January loan applications details and status that the data set is managed by the loan department; when another data set is managed by retail middle office that contains personal information records of applicants in bank.

Data generation

Create First Data Set for “Loan applications”

```
holiday_date <- c(as.Date("2024-01-06"), as.Date("2024-01-07"), as.Date("2024-01-13"), a
s.Date("2024-01-14"),
                  as.Date("2024-01-20"), as.Date("2024-01-21"), as.Date("2024-01-26"),
                  as.Date("2024-01-27"), as.Date("2024-01-28"))
time_seq <- seq(from = as.Date("2024-01-02"),
               to   = as.Date("2024-01-31"),
               by   = 'day')
time_seq <- time_seq[!time_seq %in% holiday_date]

#Add application date
set.seed(2024)

ApplicationDate <- as.Date(sample(time_seq, 120, replace = TRUE))

LoanApp <- data.frame(ApplicationDate)
```

First of all, all weekends and public holiday in January need to be excluded all weekends and public holiday in January since those date are not a working day for bank. `holiday_date` is created which is a bank closing date vector & `time_seq` is a sequence indicate the time period in January. Then working date is subsetting by using `[!time_seq %in% holiday_date]` (Clemons, 2022).

To create random `ApplicationDate` for 120 observations, `set.seed` is applied and also `as.Date` is added before `sample()` to ensure the column would become date data type. At this point, `as.data.frame` is applied to create `LoanApp` data frame structure.

```
knitr::kable(LoanApp[1:5, , drop = FALSE])
```

ApplicationDate

2024-01-03

ApplicationDate

2024-01-08

2024-01-18

2024-01-24

2024-01-18

```
#Add application no
set.seed(2024)
LoanApp %<>% arrange(ApplicationDate)
LoanApp$ApplicationNo <- as.character(sort(sample(110000:150000, size = 120, replace = F
ALSE),
                                     decreasing = FALSE))
```

Initially the order of `ApplicationDate` is arranged to show the distribution of loan application among date. The `LoanApp$ApplicationNo` is set up with 6 digits and each application no. should be unique in system, so `replace = FALSE` is applied. Also, the order of application no. sorts in ascending order to indicate the application order.

```
knitr::kable(LoanApp[1:5,])
```

ApplicationDate

ApplicationNo

2024-01-02

110223

2024-01-02

110575

2024-01-02

110874

2024-01-02

110934

2024-01-03

111023

```
#Add loan type
type <- c("HomeLoan", "CarLoan", "PersonalLoan")
set.seed(20214)
LoanApp$LoanType <- as.factor(sample(type, size = 120, replace = TRUE,
                                     prob = c(0.25, 0.4, 0.35)))
```

Home loan, car loan and unsecured personal loan are main loan types for retail banking customers. `LoanApp$LoanType` is focused on these three loan type, moreover, the number of application of different loan type would be slightly different in real world. Hence, the different probability among three loan type is applied to create relatively realistic data set.

```
knitr::kable(LoanApp[1:5,])
```

ApplicationDate

ApplicationNo

LoanType

2024-01-02

110223

PersonalLoan

2024-01-02

110575

CarLoan

2024-01-02

110874

CarLoan

2024-01-02

110934

CarLoan

ApplicationDate	ApplicationNo	LoanType
2024-01-03	111023	CarLoan

```
summary(LoanApp$LoanType)
```

```
##      CarLoan      HomeLoan PersonalLoan
##           44           30           46
```

```
home <- LoanApp$LoanType == 'HomeLoan'
car <- LoanApp$LoanType == 'CarLoan'

#Add loan amount
set.seed(20214)
LoanApp$Amount <- (ifelse(home==TRUE, sample(c("300,000","400,000","500,000","600,000"),
                                             30, replace = TRUE),
                           ifelse(car==TRUE,sample(c("40,000","50,000","60,000","70,000",
                                                    "80,000"),
                                                    44, replace = TRUE), "40,000")))
LoanApp$Amount <- as.factor(LoanApp$Amount)

#Add loan tenor
set.seed(20214)
LoanApp$Tenor <- as.integer(ifelse(home==TRUE, sample(c(120,240,360), 30, replace = TRUE),
                                                    ifelse(car==TRUE, sample(c(36,48,60), 44, replace = TRUE), 36)))
```

Summary() is applied to check the count of each loan type. Three loan types should be different in loan amount and loan tenor, since the price of a real estate and a car is huge difference in real life. home & car vectors are created and represented LoanApp\$LoanType accordingly.

Also, testing in ifelse() is needed to set the return values for false element, to be simplified, unsecured personal loan amount is set to be “40,000” with tenor 36.

Hence, ifelse() is applied to test if home exists, amount “300,000”, “400,000”, “500,000” and “600,000” & tenor “120”, “240”, “360” will be random placed repeatably for 30 observations in new columns of LoanApp\$Amount & LoanApp\$Tenor .

Same as home , ifelse() is applied to test if car exists, amount “40,000”, “50,000”, “60,000”, “70,000”, “80,000” and tenor “36”, “48”, “60” will be random placed repeatably for 44 observations in new columns of LoanApp\$Amount & LoanApp\$Tenor .

If home & car is not existed, amount “40,000” & tenor “36” will be placed for unsecured personal loan in new columns of LoanApp\$Amount & LoanApp\$Tenor .

```
knitr::kable(LoanApp[1:5,])
```

ApplicationDate	ApplicationNo	LoanType	Amount	Tenor
2024-01-02	110223	PersonalLoan	40,000	36
2024-01-02	110575	CarLoan	70,000	36
2024-01-02	110874	CarLoan	60,000	60

ApplicationDate	ApplicationNo	LoanType	Amount	Tenor
2024-01-02	110934	CarLoan	40,000	36
2024-01-03	111023	CarLoan	80,000	36

```
#Add customer no.
set.seed(2024)
LoanApp$CustomerNo <- as.character(sample(330000:550000, size = 120, replace = FALSE))

#Add customer name
set.seed(2024)
name.df <- data.frame(GENDER=sample(0:1, 120, replace=TRUE),
                      ETHNICITY=sample(1:6, 120, replace=TRUE))
LoanApp$CustomerName <- randomNames(gender=name.df$GENDER,
                                    ethnicity=name.df$ETHNICITY, name.order="first.last")
```

Each customer no. should be unique in bank system, so `LoanApp$CustomerNo` is created as a new column in `LoanApp` data frame, with 6 digits in random and ranged from 330000 to 550000 that distinguished from the `LoanApp$ApplicationNo`.

On the other hand, customer name is created by using `randomNames()`. First, `name.df` is created to set the gender and ethnicity randomly placed for all 120 observations. For example, ethnicity codes represent “American Indian or Native Alaskan”, “Asian or Pacific Islander”, “Black (not Hispanic)”, “Hispanic”, “White (not Hispanic)” & “Middle-Eastern, Arabic” in package `randomNames`.

Later, `randomNames()` is applied in order to create a full name for `LoanApp$CustomerName` according to gender and ethnicity in `name.df` and also set the name order in first name and followed by last name.

```
knitr::kable(LoanApp[1:5,])
```

ApplicationDate	ApplicationNo	LoanType	Amount	Tenor	CustomerNo	CustomerName
2024-01-02	110223	PersonalLoan	40,000	36	416564	Shafeeqa, el-Habibi
2024-01-02	110575	CarLoan	70,000	36	375755	Nathan, Lopez
2024-01-02	110874	CarLoan	60,000	60	507040	Tristan, Terry
2024-01-02	110934	CarLoan	40,000	36	349871	Sidney, Bentley
2024-01-03	111023	CarLoan	80,000	36	337801	Timothy, Mitchell

```

#Add application status
status <- c("Drawdown", "Processing", "Cancel")
set.seed(202)
LoanApp$ApplicationStatus <- as.character(sample(status, size = 120, replace = TRUE,
                                                prob = c(0.7, 0.22, 0.08)))

#Add missing value in application status column
set.seed(20214)
LoanApp %<>%
  mutate(rand = runif(120, min = 0, max = 1))
LoanApp <- mutate(LoanApp, ApplicationStatus = case_when(rand >= 0.05 ~ ApplicationStatu
s))

#Exclude the rand value for inserting NA
LoanApp <- LoanApp[,1:8]

```

For the purpose of imitating the real world status, application status, which is including “Drawdown”, “Processing” & “Cancel”, is inserted to all observation randomly by setting the different proportion to create a new column `LoanApp$ApplicationStatus`.

Finally, 5% of missing values are randomly imposed in application status column by using `runif()` to create an uniform distribution of random no. Then `case_when` is set to test if `rand` is large than or equal to 0.05, values will become “NA” in application status (Data Wrangling Course Note,2024). Also, `rand` column is created from the last step, and is required to exclude from the data set by selecting the `LoanApp[,1:8]`. The synthetic data set of `LoanApp` is now created with 8 variables and 120 observations.

“Loan applications” Data Set

Loan applications data set is created with total 120 observations and 8 variables with different data type, including date, character, factor and integer. Moreover, the data set is exported to CSV file in working directory.

```
str(LoanApp)
```

```

## 'data.frame':    120 obs. of  8 variables:
## $ ApplicationDate : Date, format: "2024-01-02" "2024-01-02" ...
## $ ApplicationNo   : chr  "110223" "110575" "110874" "110934" ...
## $ LoanType        : Factor w/ 3 levels "CarLoan","HomeLoan",...: 3 1 1 1 1 3 1 1 1 1
## ...
## $ Amount          : Factor w/ 9 levels "300,000","40,000",...: 2 8 6 2 9 2 4 2 2 8
## ...
## $ Tenor            : int   36 36 60 36 36 36 48 48 60 48 ...
## $ CustomerNo       : chr   "416564" "375755" "507040" "349871" ...
## $ CustomerName     : chr   "Shafeeqa, el-Habibi" "Nathan, Lopez" "Tristan, Terry" "Si
dney, Bentley" ...
## $ ApplicationStatus: chr   "Drawdown" "Drawdown" "Drawdown" "Drawdown" ...

```

```
head(LoanApp, 10)
```

	ApplicationDate	ApplicationNo	LoanType	Am...	Te...	Customer...	CustomerName
	<date>	<chr>	<fct>	<fct>	<int>	<chr>	<chr>
1	2024-01-02	110223	PersonalLoan	40,000	36	416564	Shafeeqa, el-Habibi
2	2024-01-02	110575	CarLoan	70,000	36	375755	Nathan, Lopez

	ApplicationDate <date>	ApplicationNo <chr>	LoanType <fct>	Am... <fct>	Te... <int>	Customer... <chr>	CustomerName <chr>
3	2024-01-02	110874	CarLoan	60,000	60	507040	Tristan, Terry
4	2024-01-02	110934	CarLoan	40,000	36	349871	Sidney, Bentley
5	2024-01-03	111023	CarLoan	80,000	36	337801	Timothy, Mitchell
6	2024-01-03	111154	PersonalLoan	40,000	36	457998	Marshall, Mcveigh
7	2024-01-03	111384	CarLoan	50,000	48	363802	Peyton, Jones
8	2024-01-03	111681	CarLoan	40,000	48	452107	Janiva, Williams
9	2024-01-03	111760	CarLoan	40,000	60	414897	Tabeion, Conover
10	2024-01-03	111988	CarLoan	70,000	48	402875	Maren, Moreno

1-10 of 10 rows | 1-8 of 9 columns

```
#Export to CSV file in working directory
write.csv(LoanApp, file = "LoanApp.csv")
```

Create Second Data Set for “Retail Banking Customer”

```
#Create 5 branch codes representing 5 retail branches
set.seed(2024)
BranchCode <- as.factor(sample(c(7627, 7628, 7629, 7630, 7631), size = 120, replace =TRUE,
                                prob = c(0.3, 0.2, 0.25, 0.1, 0.15)))

#Subset the customer no. & customer name from previous data set
CustomerNo <- LoanApp$CustomerNo
CustomerName <- LoanApp$CustomerName

#Create a new data frame for Banking customer
RetailBankingCustomer <- data.frame(BranchCode, CustomerNo, CustomerName)
```

To commence with, this data set called *RetailBankingCustomer* would contain some personal information in order to get a better understanding of each customer’s financial status. Each retail customer is assigned to and followed by a retail branch. In this case, 5 branch codes are created repeatably in random representing 5 retail branches and named the vector of `BranchCode`.

Besides, the `CustomerNo` & `CustomerName` are subset from previous *LoanApp* data set in order to maintain the consistency of banking system records. Then, data frame `RetailBankingCustomer` are constructed by `BranchCode`, `CustomerNo` and `CustomerName`.

```
knitr::kable(RetailBankingCustomer[1:5,])
```

BranchCode	CustomerNo	CustomerName
7631	416564	Shafeeqa, el-Habibi
7629	375755	Nathan, Lopez
7628	507040	Tristan, Terry

BranchCode	CustomerNo	CustomerName
7628	349871	Sidney, Bentley
7629	337801	Timothy, Mitchell

```
#add customer type
type <- c("Premier Banking", "General Banking")
set.seed(2024)
RetailBankingCustomer$CustomerType <- sample(type, size = 120, replace = TRUE, prob=c
("0.4","0.6"))

RetailBankingCustomer$CustomerType <- factor(RetailBankingCustomer$CustomerType,
                                              levels = c("General Banking","Premier Banki
ng"), ordered = TRUE)
```

To create realistic synthetic data, customer type includes “Premier Banking” & “General Banking” that is also created for each customer. It is a indicator of the relationship between customer and bank and could build up customer loyalty, since Premier Banking customers trend to maintain large amount of assets in bank and engage more banking services.

Hence, `RetailBankingCustomer$CustomerType` is created and placed repeatably for all observations by slightly different proportion. In addition, `RetailBankingCustomer$CustomerType` is changed to be ordered factor, since it is an ordinal variable representing the assets level of customers that maintained in bank.

```
knitr::kable(RetailBankingCustomer[1:5,])
```

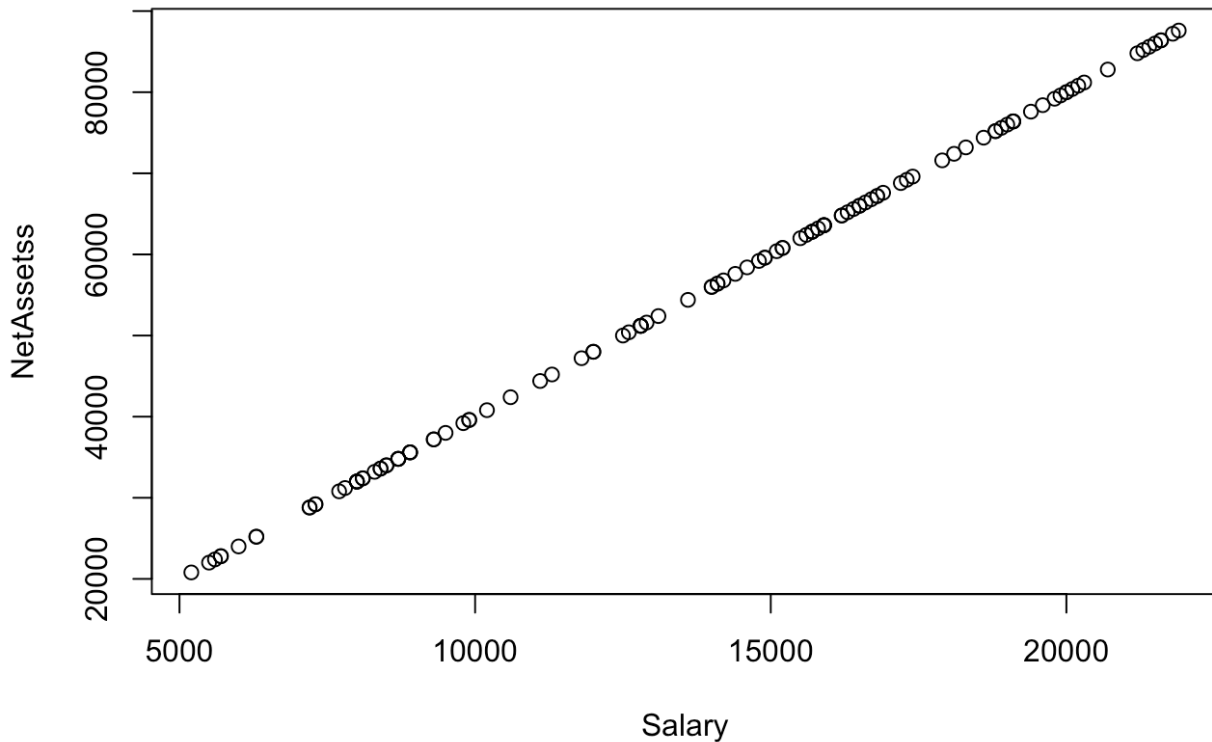
BranchCode	CustomerNo	CustomerName	CustomerType
7631	416564	Shafeeqa, el-Habibi	Premier Banking
7629	375755	Nathan, Lopez	General Banking
7628	507040	Tristan, Terry	Premier Banking
7628	349871	Sidney, Bentley	Premier Banking
7629	337801	Timothy, Mitchell	General Banking

```
#Create Correlated Random Variables
#salary <-> net_assets
set.seed(202)
correlated <- data.frame(n = 120,
                          Salary = round(runif(n = 120, min = 5000, max = 22000),-2))
set.seed(202)
correlated %<>%
  mutate(difference = rnorm(n = 120, mean = 0, sd = 3))

set.seed(202)
correlated %<>%
  mutate(NetAssets = (Salary * 4) + difference)

plot(x = correlated$Salary,
     y = correlated$NetAssets,
     main = "Salary and Net Assets",
     xlab = "Salary",
     ylab = "NetAssetss")
```

Salary and Net Assets



```
RetailBankingCustomer$Salary <- cbind(correlated$Salary)
RetailBankingCustomer$NetAssets <- cbind(correlated$NetAssets)
```

To create a pair of correlated random variables, *salary* and *net assets* are chosen, as it is assumed that the salary amount is higher, the net assets in bank trend to be larger as the same times. Moreover, both salary and net assets are vital elements of credit approval for loan in real life.

First of all, `correlated` data frame is created, meanwhile `Salary` column is also created by using `runif()` for all observations between 5,000 and 22,000, and round up to hundred. Secondly, `difference` is normally distributed with a mean of 0 and a standard deviation of 3 to represent the difference saving hobbies among all customers. Thirdly, `NetAssets` is summed by `salary amount*4 + difference` (Data Wrangling Course Note, 2024).

After creating `Salary` & `NetAssets` column, `cbind()` is used for combining those two columns into the existing `RetailBankingCustomer` data frame.

```
knitr::kable(RetailBankingCustomer[1:5,])
```

BranchCode	CustomerNo	CustomerName	CustomerType	Salary	NetAssets
7631	416564	Shafeeqa, el-Habibi	Premier Banking	7200	28796.60
7629	375755	Nathan, Lopez	General Banking	16400	65598.68
7628	507040	Tristan, Terry	Premier Banking	10600	42398.99
7628	349871	Sidney, Bentley	Premier Banking	8500	33997.46
7629	337801	Timothy, Mitchell	General Banking	11300	45199.56

```
#Add occupation for each customer
Occupation <- factor(c("Worker", "Officer","Supervisor", "Manager", "Director"), levels=
c("Worker", "Officer","Supervisor", "Manager", "Director"), order=TRUE)
set.seed(1990)
RetailBankingCustomer$Occupation <- sample(Occupation, 120, replace =TRUE,
                                           prob = c("0.07", "0.18", "0.31", "0.31", "0.
1"))

#Subset the application no. from previous data set
ApplicationNo <- LoanApp$ApplicationNo
RetailBankingCustomer <- data.frame(RetailBankingCustomer, ApplicationNo)
```

To make each customer background to become solid and reliable, occupation for each customer is also created that indicates the position code of the customer's employment stored in banking system for references.

Occupation is created as ordered factor because the salary level is usually related to the occupation level. However, salary amount would be affected by other factor, business natural also means business industry. It is reasonable that the salary amount difference of each occupation could be larger than we thought.

On the other hand, ApplicationNo is subset from previous *LoanApp* data set, since retail branch need to follow up the banking services customers applied. Then, ApplicationNo also added to data set of RetailBankingCustomer .

```
knitr::kable(RetailBankingCustomer[1:5,])
```

BranchCode	CustomerNo	CustomerName	CustomerType	Salary	NetAssets	Occupation	ApplicationNo
7631	416564	Shafeeqa, el-Habibi	Premier Banking	7200	28796.60	Manager	110223
7629	375755	Nathan, Lopez	General Banking	16400	65598.68	Officer	110575
7628	507040	Tristan, Terry	Premier Banking	10600	42398.99	Supervisor	110874
7628	349871	Sidney, Bentley	Premier Banking	8500	33997.46	Manager	110934
7629	337801	Timothy, Mitchell	General Banking	11300	45199.56	Manager	111023

```
#add missing value
set.seed(202)
RetailBankingCustomer %<>%
  mutate(random = runif(120, min = 0, max = 1))

RetailBankingCustomer %<>%
  mutate(Salary = case_when(random >= 0.06 ~ Salary))

#exclude the random column for inserting NA
RetailBankingCustomer <- RetailBankingCustomer[,1:8]
```

Finally, 5% of missing values are randomly imposed in Salary column by using runif() to create an uniform distribution of random no. Then case_when is set to test if random is large than or equal to 0.06, values will become "NA" in salary column (Data Wrangling Course Note,2024). Also, random created from last step, is required to exclude from the data set by subsetting the RetailBankingCustomer[,1:8] . The synthetic data set of RetailBankingCustomer is now created with 8 variables and 120 observations.

```
str(RetailBankingCustomer)
```

```
## 'data.frame': 120 obs. of 8 variables:
## $ BranchCode : Factor w/ 5 levels "7627","7628",...: 5 3 2 2 3 2 3 3 5 1 ...
## $ CustomerNo : chr "416564" "375755" "507040" "349871" ...
## $ CustomerName : chr "Shafeeqa, el-Habibi" "Nathan, Lopez" "Tristan, Terry" "Sidne
y, Bentley" ...
## $ CustomerType : Ord.factor w/ 2 levels "General Banking"<...: 2 1 2 2 1 2 1 1 2 1
...
## $ Salary : num [1:120, 1] 7200 16400 10600 8500 11300 18300 8400 8700 12500 15
900 ...
## $ NetAssets : num [1:120, 1] 28797 65599 42399 33997 45200 ...
## $ Occupation : Ord.factor w/ 5 levels "Worker"<"Officer"<...: 4 2 3 4 4 4 3 4 4 3
...
## $ ApplicationNo: chr "110223" "110575" "110874" "110934" ...
```

```
head(RetailBankingCustomer, 10)
```

	BranchC...	Customer...	CustomerName	CustomerType	Salary	NetAssets	Occupa
	<fct>	<chr>	<chr>	<ord>	<dbl[,1]>	<dbl[,1]>	<c
1	7631	416564	Shafeeqa, el-Habibi	Premier Banking	7200	28796.60	Man
2	7629	375755	Nathan, Lopez	General Banking	16400	65598.68	Of
3	7628	507040	Tristan, Terry	Premier Banking	10600	42398.99	Superv
4	7628	349871	Sidney, Bentley	Premier Banking	8500	33997.46	Man
5	7629	337801	Timothy, Mitchell	General Banking	11300	45199.56	Man
6	7628	457998	Marshall, Mcveigh	Premier Banking	18300	73195.71	Man
7	7629	363802	Peyton, Jones	General Banking	8400	33597.67	Superv
8	7629	452107	Janiva, Williams	General Banking	8700	34794.20	Man
9	7631	414897	Tabeion, Conover	Premier Banking	12500	50000.94	Man
10	7627	402875	Maren, Moreno	General Banking	15900	63601.41	Superv

1-10 of 10 rows | 1-8 of 9 columns

```
#export to be csv file
write.csv(RetailBankingCustomer, file = "RetailBankingCustomer.csv")
```

Merging data sets

```
report <- LoanApp %>%
  left_join(RetailBankingCustomer, by = c("ApplicationNo"="ApplicationNo", "CustomerN
o"="CustomerNo", "CustomerName"="CustomerName"))
```

By checking both *LoanApp* & *RetailBankingCustomer* data sets structure, we found that *ApplicationNo* , *CustomerNo* & *CustomerName* are existed in both data sets, so two data sets could be merged together by using *left_join()* and formed a new data frame called *report*. Therefore, we could obtain a report that includes 120 observations of loan application information in detail sorted by date in January, and followed by the personal information of the applicants.

When reading the *report*, it could be easy to identify the total no. of loan application are successfully drawdown of home loan, car loan & personal loan particularly and also identify which retail branch could achieve better performance in loan sector that obtained more number of drawdown loans.

Checking structure of combined data

```
#check data type for conversions
report %>% str()
```

```
## 'data.frame':    120 obs. of  13 variables:
## $ ApplicationDate : Date, format: "2024-01-02" "2024-01-02" ...
## $ ApplicationNo   : chr  "110223" "110575" "110874" "110934" ...
## $ LoanType        : Factor w/ 3 levels "CarLoan","HomeLoan",...: 3 1 1 1 1 3 1 1 1 1
## ...
## $ Amount          : Factor w/ 9 levels "300,000","40,000",...: 2 8 6 2 9 2 4 2 2 8
## ...
## $ Tenor           : int   36 36 60 36 36 36 48 48 60 48 ...
## $ CustomerNo      : chr   "416564" "375755" "507040" "349871" ...
## $ CustomerName    : chr   "Shafeeqa, el-Habibi" "Nathan, Lopez" "Tristan, Terry" "Si
dney, Bentley" ...
## $ ApplicationStatus: chr   "Drawdown" "Drawdown" "Drawdown" "Drawdown" ...
## $ BranchCode       : Factor w/ 5 levels "7627","7628",...: 5 3 2 2 3 2 3 3 5 1 ...
## $ CustomerType     : Ord.factor w/ 2 levels "General Banking"<...: 2 1 2 2 1 2 1 1 2
1 ...
## $ Salary           : num  [1:120, 1] 7200 16400 10600 8500 11300 18300 8400 8700 1250
0 15900 ...
## $ NetAssets        : num  [1:120, 1] 28797 65599 42399 33997 45200 ...
## $ Occupation       : Ord.factor w/ 5 levels "Worker"<"Officer"<...: 4 2 3 4 4 4 3 4 4
3 ...
```

```
report %>% head(.,8)
```

	ApplicationDate	ApplicationNo	LoanType	Am...	Te...	Customer...	CustomerName
	<date>	<chr>	<fct>	<fct>	<int>	<chr>	<chr>
1	2024-01-02	110223	PersonalLoan	40,000	36	416564	Shafeeqa, el-Habibi
2	2024-01-02	110575	CarLoan	70,000	36	375755	Nathan, Lopez
3	2024-01-02	110874	CarLoan	60,000	60	507040	Tristan, Terry
4	2024-01-02	110934	CarLoan	40,000	36	349871	Sidney, Bentley
5	2024-01-03	111023	CarLoan	80,000	36	337801	Timothy, Mitchell
6	2024-01-03	111154	PersonalLoan	40,000	36	457998	Marshall, Mcveigh
7	2024-01-03	111384	CarLoan	50,000	48	363802	Peyton, Jones
8	2024-01-03	111681	CarLoan	40,000	48	452107	Janiva, Williams

8 rows | 1-8 of 14 columns

```
#amend the column from vector to numeric
report$Salary <- as.numeric(report$Salary)
report$NetAssets <- as.numeric(report$NetAssets)

#amend tenor to be an ordered factor
report$Tenor %>% unique()
```

```
## [1] 36 60 48 360 240 120
```

```
report$Tenor <- factor(report$Tenor, levels = c("36","48","60","120","240","360"), ordered = TRUE)
```

Before data type conversions, using `str()` and `head()` to check the data types in `report`. Then, both `Salary` and `NetAssets` columns is required to amend from vector to numeric by using `as.numeric()`. Also, applying `unique()` to check the unique value in `Tenor` column and rearrange the levels and set the order by `factor()`.

Generate summary statistics

```
#Group by Loan Type, the distribution of salary amount
report %>% group_by(LoanType) %>%
  summarise(First_quantile = quantile(Salary, 0.25, na.rm = TRUE),
            Mean = mean(Salary, na.rm = TRUE),
            Median = median(Salary, na.rm = TRUE),
            Third_quantile = quantile(Salary, 0.75, na.rm = TRUE),
            SD = sd(Salary, na.rm = TRUE),
            No_of_App = n(),
            missing = sum(is.na(Salary)))
```

LoanType <fct>	First_quantile <dbl>	Mean <dbl>	Med... <dbl>	Third_quantile <dbl>	SD <dbl>	No_of_A... <int>	missing <int>
CarLoan	8700	13653.66	12800	16600	4547.862	44	3
HomeLoan	12900	15482.76	16700	19100	4485.856	30	1
PersonalLoan	9200	13900.00	14700	16375	4483.147	46	2

3 rows

In order to know about the performance among three loan types, `group_by()` is applied to `LoanType`, it could find out the statistics summary of each loan type. Then, `summarise()` the salary amount for 120 customers. In statistics, first and third quantiles indicate the 25% and 75% of probability distribution of customers applying in different type of loan, so those have to be calculated separately. Also, `na.rm = TRUE` is adopted in each calculation to make sure all missing value would be removed before calculation since `sum(is.na())` is used to find out missing value of salary amount in `report` data set.

Comparing to the median salary amount of Car Loan & Personal Loan, the median salary amount of Home Loan is relatively higher than other two types of loan. This is consistent with the real life situation since the loan amount of Home Loan is relatively higher and the requirement of strong repayment ability is demanding to address the variability of interest rate.

Scanning data

```
#scan missing value
colSums(is.na(report))
```

```
##      ApplicationDate      ApplicationNo      LoanType      Amount
##              0              0              0              0
##              Tenor      CustomerNo      CustomerName ApplicationStatus
##              0              0              0              6
##      BranchCode      CustomerType      Salary      NetAssets
##              0              0              6              0
##      Occupation
##              0
```

```
#convert the missing value to "NoResponse" in ApplicationStatus
report$ApplicationStatus %<>% replace_na("NoResponse")
```

```
#replace the missing value by salary mean
report$Salary <- impute(report$Salary, fun = mean)
```

```
#convert the data type of ApplicationStatus to be factor
report$ApplicationStatus <- as.factor(report$ApplicationStatus)
```

```
#check again the report data set
report %>% summary()
```

```
##
##      6 values imputed to 14214.04
```

```
## ApplicationDate      ApplicationNo      LoanType      Amount
## Min.      :2024-01-02      Length:120      CarLoan      :44      40,000 :54
## 1st Qu.:2024-01-08      Class :character      HomeLoan      :30      70,000 :16
## Median :2024-01-16      Mode  :character      PersonalLoan:46      500,000:12
## Mean    :2024-01-16                                     50,000 : 9
## 3rd Qu.:2024-01-24                                     300,000: 7
## Max.    :2024-01-31                                     80,000 : 7
##                                                    (Other):15
## Tenor      CustomerNo      CustomerName      ApplicationStatus BranchCode
## 36 :62      Length:120      Length:120      Cancel      : 8      7627:40
## 48 :16      Class :character      Class :character      Drawdown     :85      7628:28
## 60 :12      Mode  :character      Mode  :character      NoResponse: 6      7629:26
## 120:11                                     Processing:21      7630: 9
## 240: 6                                     7631:17
## 360:13
##
## CustomerType      Salary      NetAssets      Occupation
## General Banking:68      Min.    : 6300      Min.    :20798      Worker      : 9
## Premier Banking:52      1st Qu.: 9875      1st Qu.:35599      Officer     :16
##                               Median :14500      Median :58004      Supervisor:35
##                               Mean    :14214      Mean    :55137      Manager     :53
##                               3rd Qu.:17325      3rd Qu.:69299      Director    : 7
##                               Max.    :21900      Max.    :87601
##
```

```
#export to be csv file
write.csv(report, file = "Report.csv")
```

Regarding to scan the missing value in whole data frame, `colSums(is.na())` is applied and the total number of missing value in each column would be displayed. As there are 6 missing values found in `ApplicationStatus` and `Salary` accordingly.

Application status in bank system would remain NA while no response from customer, hence the missing value could be simply replaced by `replace_na()` to inserting "NoResponse".

Moreover, salary column is numeric data type, `impute(., fun = mean)` could be applied to compute the average salary amount of all the customer and inserted to those six missing value in order to complete the data set.

As the result, `summary()` is applied to final check if those missing values are handled properly, and also, summarising data are provided that the *frequency* of each factor columns and statistical data with *mean*, *median*, and *range* for each numeric columns. Besides, the *report* data frame is exported to be csv file.

Reference list

Wickham H, Averick M, Bryan J, Chang W, McGowan LD, François R, Golemund G, Hayes A, Henry L, Hester J, Kuhn M, Pedersen TL, Miller E, Bache SM, Müller K, Ooms J, Robinson D, Seidel DP, Spinu V, Takahashi K, Vaughan D, Wilke C, Woo K, Yutani H (2019). "Welcome to the tidyverse." *Journal of Open Source Software*, 4(43), 1686. doi:10.21105/joss.01686 (doi:10.21105/joss.01686) <https://doi.org/10.21105/joss.01686> (<https://doi.org/10.21105/joss.01686>).

Bache S, Wickham H (2022). *magrittr: A Forward-Pipe Operator for R*. R package version 2.0.3, <https://CRAN.R-project.org/package=magrittr> (<https://CRAN.R-project.org/package=magrittr>).

Wickham H, François R, Henry L, Müller K, Vaughan D (2023). *dplyr: A Grammar of Data Manipulation*. R package version 1.1.4, <https://CRAN.R-project.org/package=dplyr> (<https://CRAN.R-project.org/package=dplyr>).

Xie Y (2023). *knitr: A General-Purpose Package for Dynamic Report Generation in R*. R package version 1.45, <https://yihui.org/knitr/> (<https://yihui.org/knitr/>).

Yihui Xie (2015) *Dynamic Documents with R and knitr*. 2nd edition. Chapman and Hall/CRC. ISBN 978-1498716963

Yihui Xie (2014) *knitr: A Comprehensive Tool for Reproducible Research in R*. In Victoria Stodden, Friedrich Leisch and Roger D. Peng, editors, *Implementing Reproducible Computational Research*. Chapman and Hall/CRC. ISBN 978-1466561595

Damian W. Betebenner (2021). *randomNames: Function for Generating Random Names and a Dataset*. (R package version 1.5-0.0 URL <https://cran.r-project.org/package=randomNames> (<https://cran.r-project.org/package=randomNames>))

Harrell Jr F (2024). *Hmisc: Harrell Miscellaneous*. R package version 5.1-2, <https://CRAN.R-project.org/package=Hmisc> (<https://CRAN.R-project.org/package=Hmisc>).

Data Wrangling Course Note (2024), *Generating Synthetic Data*, course website, accessed 27 April 2024. <http://rare-phoenix-161610.appspot.com/secured/demos/Generating-Synthetic-Data.html> (<http://rare-phoenix-161610.appspot.com/secured/demos/Generating-Synthetic-Data.html>)

Clemsang (2022), *Generating a random date with exceptions in R*, Stackoverflow website, accessed 27 April 2024. <https://stackoverflow.com/questions/74799432/generating-a-random-date-with-exceptions-in-r> (<https://stackoverflow.com/questions/74799432/generating-a-random-date-with-exceptions-in-r>)

Data Wrangling Course Note (2024), *Module 5 Scan: Missing Values*, course website, accessed 28 April 2024. [http://rare-phoenix-161610.appspot.com/secured/Module_05.html#Replace_the_missing_value\(s\)_with_the_mean,_median_or_mode](http://rare-phoenix-161610.appspot.com/secured/Module_05.html#Replace_the_missing_value(s)_with_the_mean,_median_or_mode) ([http://rare-phoenix-161610.appspot.com/secured/Module_05.html#Replace_the_missing_value\(s\)_with_the_mean,_median_or_mode](http://rare-phoenix-161610.appspot.com/secured/Module_05.html#Replace_the_missing_value(s)_with_the_mean,_median_or_mode))

Applied Analytics Module 2 Note (2024), *Module 2 Descriptive Statistics through Visualisation*, course website, accessed 18 April 2024. https://astral-theory-157510.appspot.com/secured/MATH1324_Module_02.html (https://astral-theory-157510.appspot.com/secured/MATH1324_Module_02.html) (https://astral-theory-157510.appspot.com/secured/MATH1324_Module_02.html) (https://astral-theory-157510.appspot.com/secured/MATH1324_Module_02.html)